

Implementación de Auditoría de Operaciones

Actividad N°: 26 **Fecha:** 06/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Completar el módulo de auditoría (RF-08), asegurando que toda operación relevante quede registrada de forma automática y consultable, sin acoplar esa responsabilidad a la lógica de negocio de cada controlador.

2. Middleware `auditLogger` (registro automático desacoplado)

`backend/src/middleware/auditLogger.js` implementa un patrón de **interceptor de respuesta**: envuelve `res.json` para registrar el log de auditoría **después** de que la operación principal ya respondió exitosamente, sin bloquear al usuario:

```
const auditLogger = (tipo) => (req, res, next) => {
  const originalJson = res.json;
  res.json = function(data) {
    if (res.statusCode < 400 && req.user) {
      setImmediate(async () => {
        // construir logData según "tipo" y crear AuditLog
        await AuditLog.create(logData);
      });
    }
    return originalJson.call(this, data);
  };
  next();
};
```

Decisiones de diseño clave:

- Se usa `setImmediate` para que la escritura del log ocurra en el siguiente ciclo del event loop, **después** de enviar la respuesta HTTP al cliente — el usuario no espera al log para recibir su confirmación de canje/impresión.
- Solo se registra si `res.statusCode < 400`, es decir, únicamente operaciones exitosas.
- El middleware se reutiliza en distintas rutas simplemente parametrizando el `tipo` ('canje', 'canje_masivo', 'impresion', 'reimpresion', 'creacion_usuario'), sin duplicar código de registro en cada controlador.

3. Endpoints de consulta de auditoría (`auditController.js`)

Endpoint	Función
GET /api/audit	Lista paginada de logs, con filtros por <code>tipo</code> , <code>usuario</code> , <code>ticketId</code> y rango de fechas (<code>fechaInicio/fechaFin</code>)

Endpoint	Función
GET <code>/api/audit/summary</code>	Resumen agregado: logs por tipo, logs por usuario (con <code>\$lookup</code> a <code>users</code> para traer nombre y rol), logs por día

Ambos endpoints están restringidos a `authorize('jefe')`.

4. Doble vía de registro de auditoría

Se identifican dos mecanismos complementarios que coexisten en el sistema:

1. **Vía middleware genérico** (`auditLogger`): usado en rutas de tickets y creación de usuarios, captura automáticamente el contexto del request (`req.body`, `req.user`).
2. **Vía registro manual dentro del controlador** (`AuditLog.create(...)` directo): usado en `authController.js` para `login`, `logout` y `cambio_password`, y en `canjeTicket/bulkCanjeTickets` para incluir detalles de negocio más específicos (ej. `parentesco`, `quienOtro`, `bulkOperation`).

Ambas vías escriben sobre el mismo modelo `AuditLog`, por lo que la consulta y el resumen (`getAuditLogs/getAuditSummary`) funcionan de manera uniforme sin importar cuál mecanismo generó cada registro.

5. Frontend — `AuditPage.jsx`

- Tabla de logs con columnas: tipo de operación, usuario (nombre y rol vía `populate`), ticket/transacción afectada, punto de trabajo y fecha.
- Filtros por tipo de operación y por usuario.
- Acceso exclusivo para el rol Jefe (protegido en frontend y backend).

6. Conclusiones del día

El módulo de auditoría queda completo: cada operación sensible del sistema (autenticación, gestión de usuarios, canje, impresión) genera un registro trazable, ya sea vía el middleware genérico o vía registro manual con detalle de negocio, y ambos son consultables desde los mismos endpoints de reporte.

Observaciones: Sin observaciones.